

Foundations of 4D/6D Object Capture for Virtual Environments

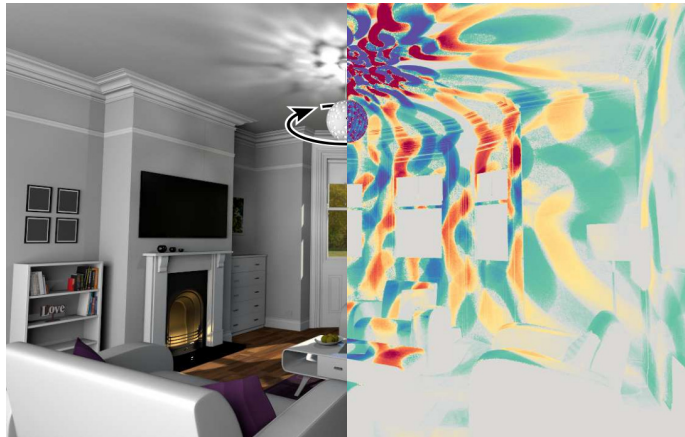


Deep Learning



March 5, 2026

→ **Differentiable Rendering**



Today: Deep Learning

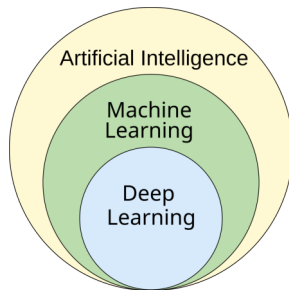
Question: What is "Machine Learning"?

*"A computer program is said to learn from **experience** E with respect to some class of **tasks** T and **performance measure** P , if its performance at tasks in T , as measured by P , improves with experience E ." (Mitchell¹)*

In the context of Differentiable Rendering:

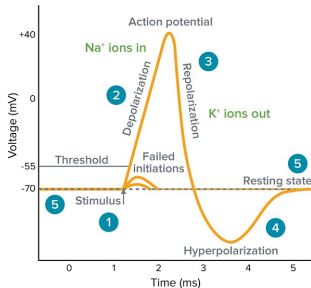
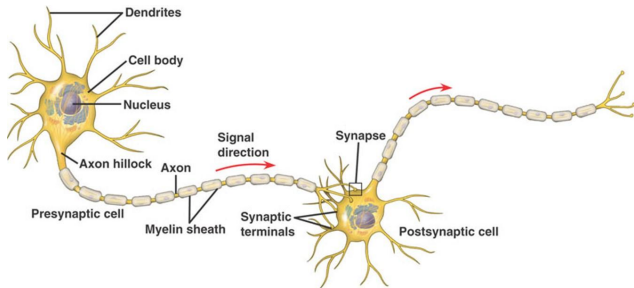
- ▶ **Experience:** Training on input images
- ▶ **Task:** Rendering
- ▶ **Performance:** Difference between rendered images and input images

→ Deep Learning: Machine Learning with Neural Networks



¹T. M. Mitchell. *Machine Learning*. Vol. 1. 9. McGraw-hill New York, 1997.

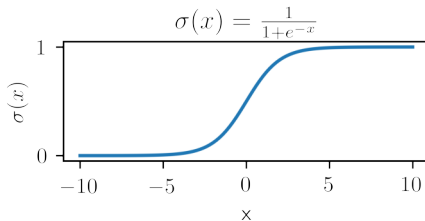
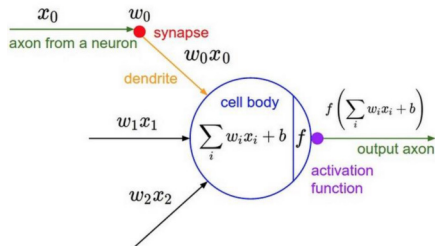
Biological Inspiration: How is information processed in the brain?



Signal Processing in Neurons:

1. Signal starts as "discrete firing event" at axon hillock
2. Signal gets transferred over axon to synapses
3. Synapses propagate signal to dendrites of next neuron
4. Dendrites receive signal and forward it to cell body → only if signal surpasses certain threshold

Question: How can we model this computationally?



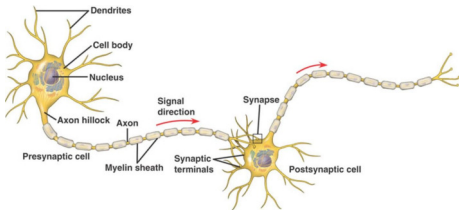
Output ("firing rate") of 1 neuron with D input values x_i , synaptic weights w_i , bias b :

1. First, the activation ("membrane potential") is calculated:

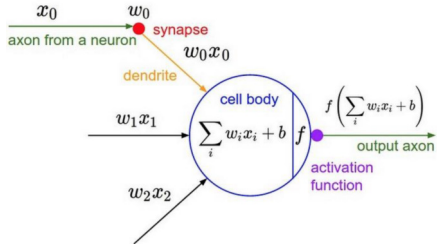
$$\underbrace{a}_{\text{activation}} = \sum_{i=1}^D w_i \cdot x_i + \underbrace{b}_{\text{bias}}$$

2. Then, an **activation function** $f(\cdot)$ is applied to calculate the output $z = f(a)$ of the neuron
 → Should be **non-linear** and **differentiable** (e.g. sigmoid function σ)

Comparison:



VS.



Similarities

- ▶ Synapses and synaptic weights
- ▶ Membrane potential / activation function
- ▶ Levels of abstraction

Differences

- ▶ Neuron output (Individual spikes vs. Firing rate)
- ▶ Learning process (Hebbian learning vs. Training with gradient descent)
- ▶ Connections between neurons (Recurrent vs. Feed forward)

Usually, several neurons are applied to the same input and summarised in a **layer of size M**

$$\underbrace{a_j}_{\text{activation}} = \sum_{i=1}^D w_{ji} \cdot x_i + \underbrace{b_j}_{\text{bias}} \quad j = 1, \dots, M$$
$$z_j = f(a_j)$$

This can be written more compactly with the help of vectors and matrices:

$$\mathbf{a} = \mathbf{W} \mathbf{x} + \mathbf{b} \quad , \quad \mathbf{z} = f(\mathbf{a})$$

- ▶ $\mathbf{W} \in \mathbb{R}^{M \times D}$ contains the synaptic weights w_{ji}
- ▶ $\mathbf{b} \in \mathbb{R}^M$ contains the bias values b_i

Alternative: The activation $\mathbf{a}$ can also be expressed with a single linear transformation in homogeneous coordinates:

$$\mathbf{a} = \left[\begin{array}{c|c} \mathbf{W} & \mathbf{b} \\ \hline \mathbf{0} & 1 \end{array} \right] \cdot \begin{pmatrix} \mathbf{x} \\ 1 \end{pmatrix} = \hat{\mathbf{W}} \hat{\mathbf{x}}$$

However, a single layers is also uncommon. A neural network usually contains a **large number of layers N** which are applied one after the other:

$$a^{(1)} = W^{(1)} \cdot x + b^{(1)}$$

$$a^{(2)} = W^{(2)} \cdot z^{(1)} + b^{(2)}$$

$$\vdots$$

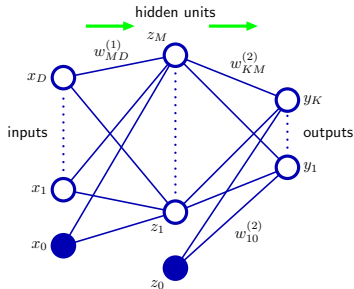
$$a^{(N)} = W^{(N)} \cdot z^{(N-1)} + b^{(N)}$$

$$z^{(1)} = f^{(1)}(a^{(1)})$$

$$z^{(2)} = f^{(2)}(a^{(2)})$$

$$\vdots$$

$$y = f^{\text{out}}(a^{(N)})$$



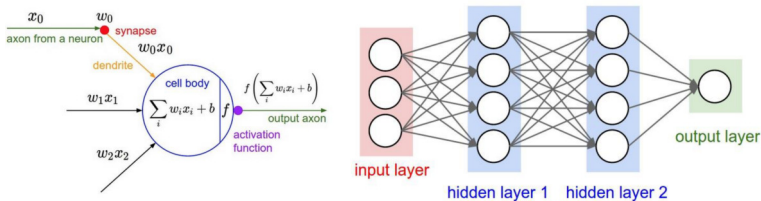
Types of layers:

- Input: x
- Hidden: Intermediate results $z^{(k)}$
- Output: y

→ Counting of layers not always consistent in literature!

Example: This can be called a ...

- 3-layer network
- single-hidden-layer network
- 2-layer network
(→ number of multiplications $W^{(i)}$)



So, we can understand a neural network as a parameterised function

$$\text{MLP}_{\Theta}(x) = f^{\text{out}}(\mathbf{W}^{(N)} \cdot f^{(N-1)}(\dots \mathbf{W}^{(2)} \cdot f^{(1)}(\mathbf{W}^{(1)} \cdot x + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)} \dots) + \mathbf{b}^{(N)})$$

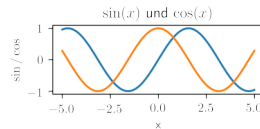
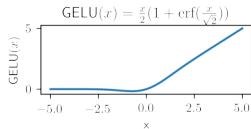
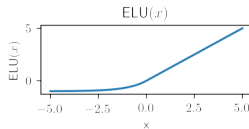
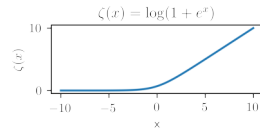
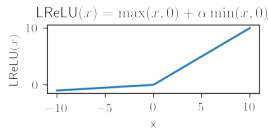
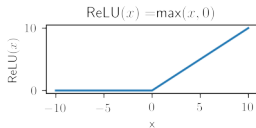
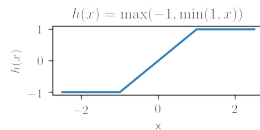
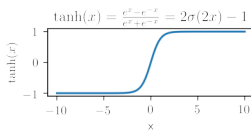
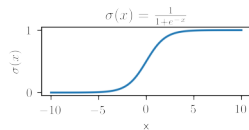
with learnable parameters

$$\Theta = (\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{W}^{(2)}, \mathbf{b}^{(2)}, \dots, \mathbf{W}^{(N)}, \mathbf{b}^{(N)})^T$$

→ Also called **Multi-Layer Perceptrons (MLP)**

→ Interactive Demo for MLPs ←

There exists a lot of different activation functions:



→ Popular choice: ReLU (but others may work better depending on context)

Question: Why is it important that the activations $f(\cdot)$ are non-linear?

Suppose we have a network with N hidden layers with linear activations $f(\cdot)$ (in homogeneous coordinates):

$$\begin{aligned}\text{MLP}_{\Theta}(\mathbf{x}) &= f^{\text{out}}(\hat{\mathbf{W}}^{(N)} \cdot f^{(N-1)} \dots f^{(2)}(\hat{\mathbf{W}}^{(2)} \cdot f^{(1)}(\hat{\mathbf{W}}^{(1)} \cdot \hat{\mathbf{x}}))) \\ &= f^{\text{out}}(\hat{\mathbf{W}}^{(N)} \cdot f^{(N-1)} \dots f^{(2)}(\hat{\mathbf{W}}^{(2)} \cdot \hat{\mathbf{W}}_f^{(1)} \cdot \hat{\mathbf{x}})) && \left[\text{New weights } \hat{\mathbf{W}}_f^{(1)} := f^{(1)} \circ \hat{\mathbf{W}}^{(1)} \right] \\ &= f^{\text{out}}(\underbrace{\hat{\mathbf{W}}_f^{(N)} \dots \hat{\mathbf{W}}_f^{(2)} \cdot \hat{\mathbf{W}}_f^{(1)}}_{\hat{\mathbf{W}}'} \cdot \hat{\mathbf{x}}) && \left[\text{New weights } \hat{\mathbf{W}}_f^{(2)}, \dots, \hat{\mathbf{W}}_f^{(N-1)} \right] \\ &= f^{\text{out}}(\hat{\mathbf{W}}' \cdot \hat{\mathbf{x}})\end{aligned}$$

→ For every network with only linear activations, we can find an equivalent collapsed network

→ Interactive Demo (choose "Linear") ←

Theorem (Universal Approximation Theorem)

Given a continuous function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ and any $\varepsilon > 0$, there exists a neural network function

$$g(\mathbf{x}) = \sum_{i=1}^N c_i \sigma(\mathbf{w}_i^T \cdot \mathbf{x} + b_i)$$

such that

$$|f(\mathbf{x}) - g(\mathbf{x})| < \varepsilon \quad \forall \mathbf{x} \in K \subseteq \mathbb{R}^d$$

where

- ▶ σ is a non-constant, bounded, and continuous activation function
- ▶ $\mathbf{W} = [\mathbf{w}_1 \mid \dots \mid \mathbf{w}_N]^T \in \mathbb{R}^{N \times d}$, $\mathbf{b} = (b_i) \in \mathbb{R}^N$, $\mathbf{c} = (c_i) \in \mathbb{R}^N$ are learned parameters

In other words: A neural network with one hidden layer can approximate any continuous f on some $K \subseteq \mathbb{R}^n$

→ Only theoretical result: Does not provide efficient way to find parameters and N may become *extremely large*

Example:

Given: Uniformly sampled points from functions

(a) $f(x) = x^2$

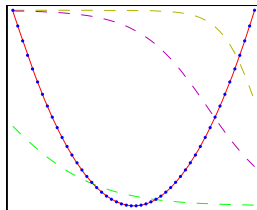
(b) $f(x) = \sin(x)$

(c) $f(x) = |x|$

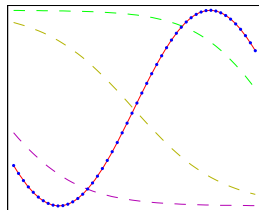
(d) $f(x) = \underbrace{H(x)}_{\text{Heaviside step}} := \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$

Output: Learned function by MLP

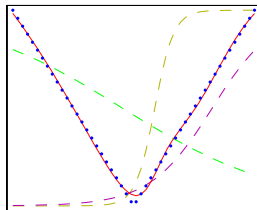
- ▶ 1 hidden layer with 3 hidden units and tanh activation
- ▶ Linear output unit f^{out}



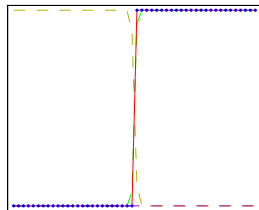
(a)



(b)



(c)



(d)

Other curves show output of each of the 3 hidden units

How can we find suitable parameters Θ ? $\rightarrow$ Training

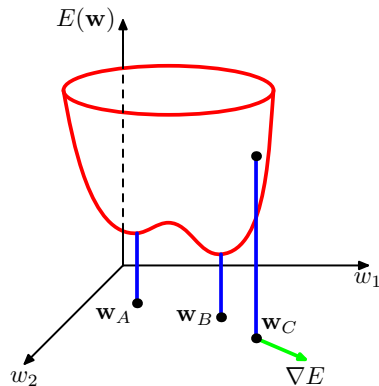
Ingredients:

- ▶ Input data x
- ▶ Loss function $\mathcal{L}$
- ▶ Optimizer

Paradigms:

- ▶ Supervised Learning: Learn to fit to desired output data y of x
 $\rightarrow$ Primary focus
- ▶ Unsupervised Learning: No explicit output provided, instead find patterns in x
- ▶ Self-supervised Learning: Similar to unsupervised, but pseudo output data $\hat{y}$ is generated from input x

$\rightarrow$ In any case: We need input data x , ideally a lot of it!

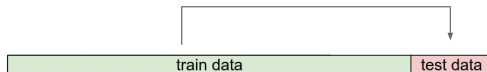


Problem: Despite the learnable parameters, there are other hyperparameters to choose during optimization

- ▶ Step size of t_k of optimizer
- ▶ Weighting between multiple losses → Single loss typically not enough
- ▶ etc.

Idea: Split input data into train and test sets

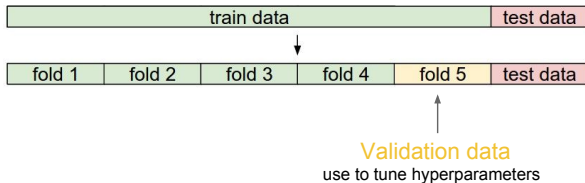
Try out what hyperparameters work best on test set.



→ Do **NOT** use test data for any training-related stuff, only for evaluating generalization performance!

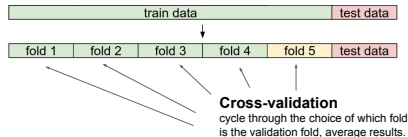
More precisely, split data into train, validation, and test sets

- ▶ Train data: For optimizing the neural network
- ▶ Validation data: For tuning hyperparameters and detect over-/underfitting
- ▶ Test data: Solely for evaluating performance



"Rule of thumb":

1. Split entire data into 80% train + 20% test
2. Split train data into 80% actual train + 20% validation
3. (Optional) Cross-validation: Cycle through train-validation splits → Expensive

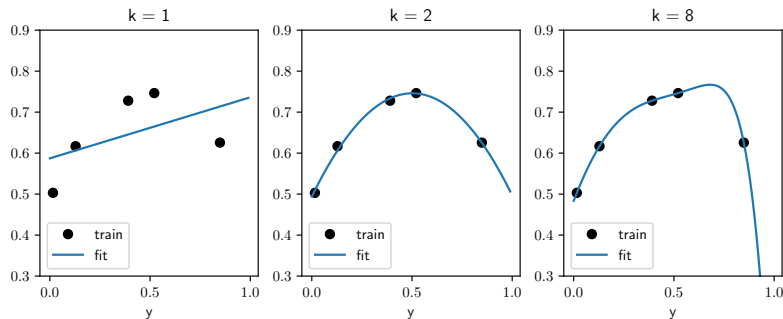


Question: How can we find out if a learned network performs "good"?

Example:

Instead of a MLP, assume a polynomial of degree k :

$$f_{\Theta}(x) = \sum_{n=0}^k \theta_n x^n$$

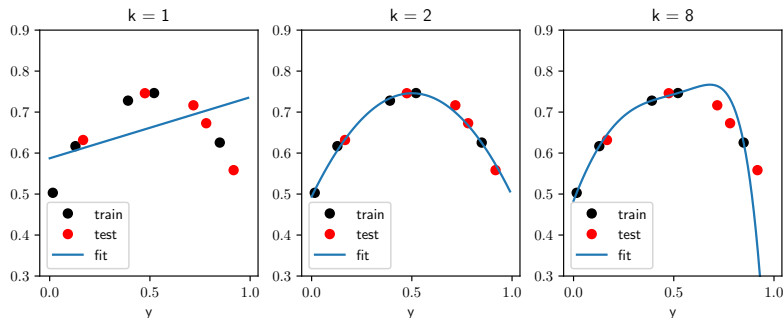


→ $k = 1$ is bad, but $k = 2$ and $k = 8$ both fit to all inputs

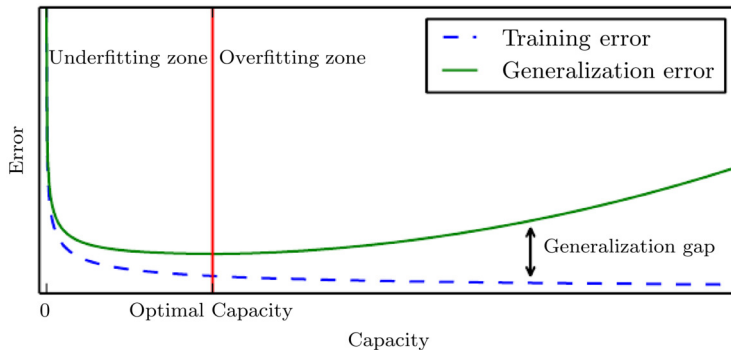
Underfitting and Overfitting: The performance of a machine learning algorithm depends on 2 factors:

1. Small training error
2. Small performance difference between training and test errors (Generalization gap)

Algorithms that **underfit** cannot fulfil the first criterion; algorithms that **overfit** can fulfil the first but not the second criterion.



Underfitting and Overfitting:



- ▶ Neural Networks can often achieve small training errors and small generalization errors
- ▶ Use validation set to detect overfitting

Idea: Use Gradient Descent to find parameters of a MLP:

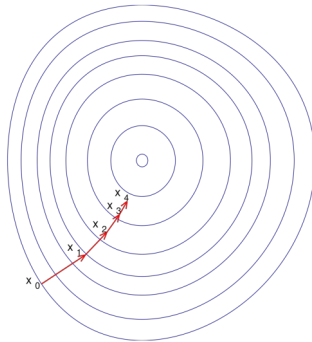
$$\Theta_{k+1} = \Theta_k - \epsilon \cdot \nabla \mathcal{L}(\Theta_k)$$

→ In Deep Learning context, step size ϵ is called "learning rate"!

Problem: Each of m inputs contributes to the gradient via chain rule:

$$\mathcal{L}(\Theta) = \sum_{i=1}^m \rho(r_i(\Theta))$$

→ There can be millions of inputs i



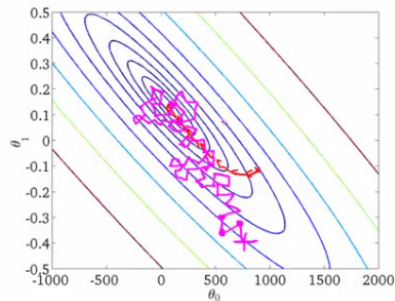
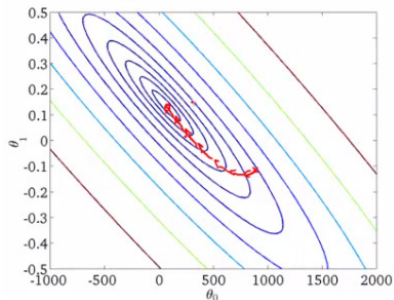
Stochastic Gradient Descent: Compute gradient on subsets of data (size 1 or larger), called "mini-batches"

$$\Theta_{k+1} = \Theta_k - \epsilon \cdot \nabla \mathcal{L}(\Theta_k)$$

$$= \Theta_k - \epsilon \cdot \sum_{i=1}^m \nabla \underbrace{\mathcal{L}_i}_{\rho(r_i(\cdot))}(\Theta_k)$$

$\Rightarrow$

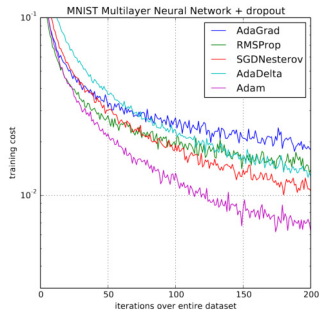
$$\Theta_{k+1} = \Theta_k - \epsilon \cdot \nabla \mathcal{L}_i(\Theta_k)$$



In practice, successors of Stochastic Gradient Descent are typically used, such as:²

THE de-facto standard optimizer: **Adam**

- ▶ SGD Nesterov (with Momentum)
- ▶ RMSprop
- ▶ Adadelta
- ▶ Adagrad
- ▶ ...



> 238k Citations (as of March 2026)

²D. P. Kingma and J. Ba. "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations (ICLR)*. 2015.

Idea: Instead of directly following the **noisy** negative gradient directions, estimate the first and second moments of the gradient.

Definition (Raw Moment)

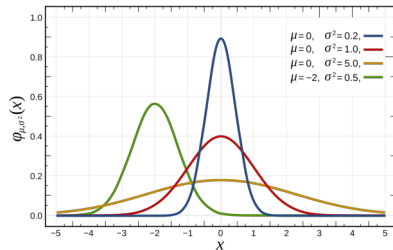
Given a random variable X with probability density function $p : \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}$, the k -th raw moment is defined as:

$$\mu_k = \mathbb{E}[X^k] = \int_{-\infty}^{\infty} x^k p(x) dx$$

Interpretation:

	Mathematics	Physics
1st (Raw) Moment	Expected value	Center of mass
2nd (Central) Moment	Variance	Rotational inertia

(Familiar) Problem: How to compute moments if only current/last gradient is known?



(Familiar) Problem: How to compute moments if only current/last gradient is known?

Lemma (Exponential Moving Average)

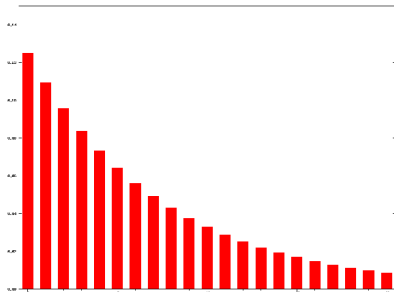
Let x_0 be some initial value, $x_1, x_2, \dots, x_{n+1}$ be a sequence of samples, and $\alpha \in [0, 1]$. Then the weighted average

$$X_{n+1} = \sum_{i=0}^{n+1} (1 - \alpha) \alpha^{n+1-i} \cdot x_i$$

can be computed online using the formula

$$X_{n+1} = \alpha \cdot X_n + (1 - \alpha) \cdot x_{n+1}$$

and the initialization $X_0 = x_0$.



- Low-pass filter on samples
- Automatically weights down all previous samples when new one arrives

Algorithm:

Algorithm 8.7 The Adam algorithm

Require: Step size ϵ (Suggested default: 0.001)

Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$.
(Suggested defaults: 0.9 and 0.999 respectively)

Require: Small constant δ used for numerical stabilization. (Suggested default: 10^{-8})

Require: Initial parameters θ

Initialize 1st and 2nd moment variables $\mathbf{s} = \mathbf{0}$, $\mathbf{r} = \mathbf{0}$

Initialize time step $t = 0$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

$t \leftarrow t + 1$

Update biased first moment estimate: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g}$

Update biased second moment estimate: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g}$

Correct bias in first moment: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}$

Correct bias in second moment: $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$

Compute update: $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$ (operations applied element-wise)

Apply update: $\theta \leftarrow \theta + \Delta \theta$

end while

The different state variables of Adam contain:

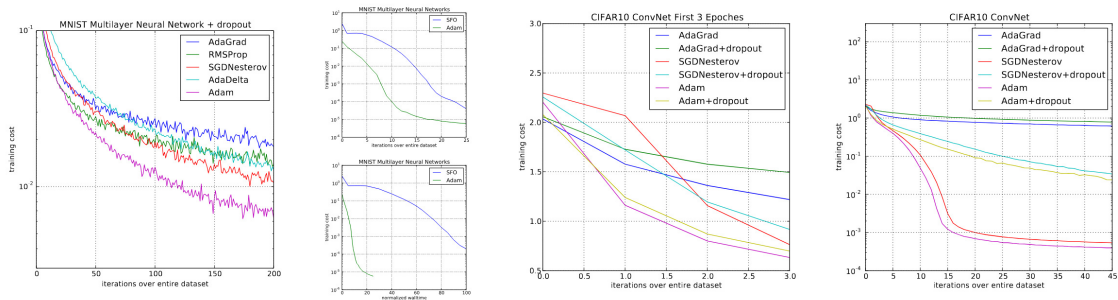
- ▶ $\mathbf{s}$: 1st moment of gradients $\mathbf{g}$ with EMA
- ▶ $\mathbf{r}$: 2nd moment of gradients $\mathbf{g}$ with EMA
- ▶ $\hat{\mathbf{s}}, \hat{\mathbf{r}}$: Bias corrected terms, due to:

$$\mathbb{E}[\mathbf{s}] = \mathbb{E}[\mathbf{g}] \cdot (1 - \rho_1^t) + c_1$$

$$\mathbb{E}[\mathbf{r}] = \mathbb{E}[\mathbf{g}^2] \cdot (1 - \rho_2^t) + c_2$$

→ "Standard" choice: $\rho_1 = 0.9$, $\rho_2 = 0.999$

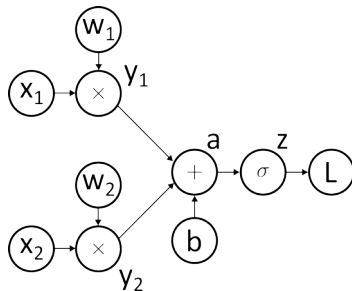
Results:



Since we now have a good optimizer, we just need to compute gradients

→ Reverse-mode Automatic Differentiation!

Note: In Deep Learning, the application of this approach to neural networks is called "backpropagation"



Example: Single neuron

$$\frac{\partial \mathcal{L}}{\partial z} \Rightarrow \boxed{\frac{\partial \mathcal{L}}{\partial a}} = \frac{\partial \mathcal{L}}{\partial z} \frac{\partial z}{\partial a} \Rightarrow \begin{aligned} \boxed{\frac{\partial \mathcal{L}}{\partial b}} &= \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial b} \\ \frac{\partial \mathcal{L}}{\partial y_i} &= \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial y_i} \end{aligned} \Rightarrow \begin{aligned} \boxed{\frac{\partial \mathcal{L}}{\partial w_i}} &= \frac{\partial \mathcal{L}}{\partial y_i} \frac{\partial y_i}{\partial w_i} \\ \frac{\partial \mathcal{L}}{\partial x_i} &= \frac{\partial \mathcal{L}}{\partial y_i} \frac{\partial y_i}{\partial x_i} \end{aligned}$$

Question: What are these derivatives in particular?

$$\mathbf{a} = \mathbf{W} \mathbf{x} + \mathbf{b} \quad , \quad \mathbf{z} = f(\mathbf{a})$$

In the following, we will denote the derivatives by their Jacobians matrices: $\mathbf{J}_c(\mathbf{d}) = \frac{\partial \mathbf{c}}{\partial \mathbf{d}}$.

► Derivative w.r.t. bias $\mathbf{b} \in \mathbb{R}^M$:

$$\mathbf{J}_a(\mathbf{b}) = \mathbf{I}$$

► Derivative w.r.t. input $\mathbf{x} \in \mathbb{R}^D$:

$$\mathbf{J}_a(\mathbf{x}) = \mathbf{W}$$

► Derivative w.r.t. weights $\mathbf{W} \in \mathbb{R}^{M \times D}$:

$$\mathbf{J}_a(\mathbf{W}) = \frac{\partial a_i}{\partial w_{kl}} = \begin{cases} x_l & i = k \text{ (correct row picked)} \\ 0 & \text{otherwise} \end{cases} = \mathbf{I} \otimes \mathbf{x}$$

1. Here, we now have 3 indices (i, k, l) for $\mathbf{J}_a(\mathbf{W}) \in \mathbb{R}^{M \times M \times D}$
2. Multiplication with derivative $\mathbf{J}_L(\mathbf{a}) \in \mathbb{R}^{1 \times M}$ of the loss results in the expected $1 \times M \times D$ tensor

Question: How do the derivatives look for a MLP with several hidden layers?

$$\mathbf{a}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$\mathbf{z}^{(1)} = f^{(1)}(\mathbf{a}^{(1)})$$

Layer 1

$$\mathbf{a}^{(2)} = \mathbf{W}^{(2)}\mathbf{z}^{(1)} + \mathbf{b}^{(2)}$$

$$\mathbf{z}^{(2)} = f^{(2)}(\mathbf{a}^{(2)})$$

Layer 2

$$\mathbf{a}^{(3)} = \mathbf{W}^{(3)}\mathbf{z}^{(2)} + \mathbf{b}^{(3)}$$

$$\hat{\mathbf{y}} = f^{\text{out}}(\mathbf{a}^{(3)})$$

Output

$$\mathcal{L}(\hat{\mathbf{y}}(\mathbf{x}), \mathbf{y}) = \sum_{i=1}^m (\hat{y}_i(\mathbf{x}) - y_i)^2$$

Loss

→ Chain derivatives together, i.e. matrix multiplications of Jacobians

There are a lot of further aspects that we have not discussed here:

Architectures:

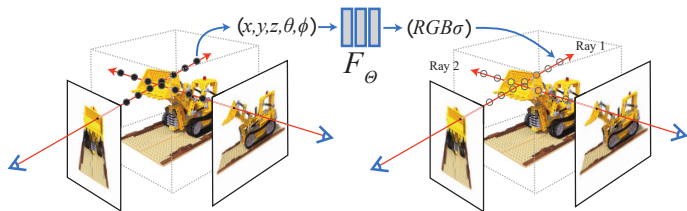
- ▶ Convolutional Neural Networks (CNN)
- ▶ Residual Neural Networks (ResNet)
- ▶ U-Net
- ▶ Autoencoders
- ▶ Generative Adversarial Networks (GAN)
- ▶ Transformers
- ▶ Diffusion Models
- ▶ ...

Applications:

- ▶ (Image) Classification
- ▶ Neural Language Processing
→ LLMs
- ▶ Novel View Synthesis
→ Neural Radiance Fields

Next Lecture: Neural Radiance Fields

→ Neural Radiance Fields



- [1] D. P. Kingma and J. Ba. “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations (ICLR)*. 2015.
- [2] T. M. Mitchell. *Machine Learning*. Vol. 1. 9. McGraw-hill New York, 1997.